SheShield

Real-Time Emergency Response System

Developed By: Asmi M Shetty

Deployed link: https://she-shield-zeta.vercel.app/
Github: https://github.com/asmishetty-sudo/SheShield
Demo video: https://youtu.be/U6oMOUTUaR8

ABSTRACT

SheShield is a real-time emergency response web application developed to provide immediate assistance to individuals in distress by connecting them with nearby volunteers. The system is designed to minimize response time during emergencies by leveraging geolocation-based matching and real-time communication technologies. Both registered users and guest users can trigger SOS alerts, ensuring accessibility even without prior authentication.

When an SOS is triggered, the system identifies volunteers within a specified geographic radius using spatial queries and instantly broadcasts alerts through socket-based real-time communication. Volunteers can accept requests, view the sender's location, and take necessary action. The platform maintains live synchronization between sender and responders, enabling continuous updates regarding response status.

To enhance reliability, SheShield incorporates an intelligent escalation mechanism. If no volunteers respond within a defined time window, the system automatically retries the alert by expanding the search radius in multiple stages. If the request remains unresolved after several attempts, it is escalated to the administrator through email notification, ensuring that critical cases are not ignored.

Additionally, the system supports trusted contact notifications, allowing emergency alerts to be sent via email to predefined contacts of the user. Backend processes are designed to operate asynchronously, enabling tasks such as alert dispatching and escalation handling to continue executing even after the server responds to the client, thereby improving system efficiency and responsiveness.

The application is built using a modern full-stack architecture, with a Next.js frontend, Node.js and Express.js backend, MongoDB database, and Socket.IO for real-time communication. Security is maintained through JWT-based authentication and encrypted password storage.

Overall, SheShield demonstrates a scalable and fault-tolerant approach to emergency response systems, emphasizing real-time interaction, reliable alert delivery, and efficient backend processing.

Table of Contents

1. INTRODUCTION 1
2. OBJECTIVES 2
3. SYSTEM ARCHITECTURE 3
4. KEY FEATURES 6
5. WORKFLOW 15
6. DATABASE DESIGN 16
7. IMPLEMENTATION DETAILS 21
8. RESULTS / OUTPUT 25
9. FUTURE IMPROVEMENTS 33
10. CONCLUSION 34
11. REFERENCES 35

1. INTRODUCTION

In emergency situations, immediate assistance can be critical in preventing harm and ensuring safety. However, many existing systems depend on delayed communication methods, centralized control, or manual intervention, which can significantly increase response time. Traditional emergency services may not always be able to respond instantly due to high demand, limited resources, or lack of real-time coordination.

SheShield is designed to address these challenges by providing a real-time, decentralized emergency response platform that connects individuals in distress with nearby volunteers. The system enables both registered users and non-registered (guest) users to trigger SOS alerts quickly and efficiently, ensuring accessibility even in urgent situations where logging in may not be feasible.

Once an SOS is triggered, the system uses geolocation-based querying to identify volunteers within a specified radius and sends alerts instantly using real-time communication protocols. Volunteers can accept the request, view the sender's location, and navigate to provide assistance. The platform maintains continuous synchronization between users and volunteers, allowing real-time updates on the status of the emergency.

To enhance system reliability, SheShield incorporates an automated escalation mechanism. If no volunteers respond within a defined time frame, the system progressively expands the search radius and retries sending alerts. In cases where the SOS remains unresolved after multiple attempts, the system escalates the situation by notifying the administrator through email, ensuring that critical emergencies are not overlooked.

Additionally, the platform supports multi-channel alerting by notifying trusted contacts through email, further increasing the chances of timely assistance. The backend is designed to handle asynchronous processes, allowing alerting and escalation tasks to continue independently without affecting the responsiveness of the system.

Overall, SheShield provides an efficient, scalable, and reliable solution for real-time emergency response by combining geolocation services, real-time communication, and intelligent backend processing.

2. OBJECTIVES

The primary objective of the SheShield system is to design and develop a reliable, real-time emergency response platform that ensures rapid communication and assistance during critical situations. The project focuses on integrating modern web technologies, real-time communication, and intelligent backend processing to create an efficient and scalable solution.

The specific objectives of the system are as follows:

- **To develop a real-time SOS alert system:**
  The system aims to enable users to trigger emergency alerts that are instantly transmitted to nearby volunteers using real-time communication technologies such as WebSockets. This ensures minimal delay in alert delivery and faster response times.
- **To connect users with nearby volunteers using geolocation:**
  The platform utilizes geospatial querying techniques to identify and notify volunteers within a specific radius of the user's location. This helps in ensuring that assistance is provided by the closest available responders.
- **To implement guest-based emergency access without authentication:**
  The system allows non-registered users to trigger SOS alerts using a temporary guest identification mechanism. This ensures accessibility in emergency situations where users may not have the time or ability to log in.
- **To design an intelligent escalation mechanism for unresponded SOS requests:**
  If an SOS alert is not accepted within a defined time frame, the system automatically retries the alert by expanding the search radius in multiple stages. This increases the probability of finding available volunteers and ensures that no request is left unattended.
- **To enable multi-channel notification delivery:**
  The platform supports sending emergency alerts through multiple communication channels, such as email and messaging services, including notifications to trusted contacts and administrators. This improves the reliability of alert delivery.
- **To ensure system reliability using asynchronous backend processing:**
  The backend is designed to handle long-running tasks such as alert retries and escalation using asynchronous mechanisms. This allows the server to respond quickly to client requests while continuing background processes independently.
- **To provide real-time status tracking and updates:**
  The system maintains live synchronization between the sender and volunteers, allowing users to view responding volunteers and volunteers to access relevant details of the emergency in real time.
- **To implement role-based system functionality:**
  The platform supports multiple user roles, including users, volunteers, and administrators, each with specific functionalities such as SOS triggering, response handling, and system management.

3. SYSTEM ARCHITECTURE

The SheShield system follows a full-stack, event-driven architecture designed to support real-time emergency communication, efficient data handling, and reliable alert delivery. The system is divided into multiple layers, each responsible for specific functionalities, ensuring scalability and maintainability.

At a high level, the system consists of the following components:
Frontend (client-side), Backend (server-side), Database, Real-time communication layer, and External notification services.

3.1. User / Guest Layer

This layer represents the end users of the system:

- Registered users (can login and manage profile)
- Guest users (can trigger SOS without authentication)
- Volunteers (respond to SOS alerts)

Users interact with the system through a web interface where they can:

- Trigger SOS alerts
- View live updates
- Accept or resolve emergencies

3.2. Frontend (Next.js)

The frontend is built using Next.js and serves as the user interface layer.

Responsibilities:

- Capturing user input (location, SOS trigger)
- Displaying real-time SOS status
- Showing volunteer responses
- Managing UI states (active SOS, resolved SOS)

How it works:

- Sends API requests to the backend
- Connects to the socket server for real-time updates
- Listens to events like:
    - sos-alert
    - sos-accepted
    - sos-resolved

3.3. Backend (Node.js / Express)

The backend is the core processing unit of the system.
Responsibilities:

- Handling API requests (trigger SOS, accept SOS, resolve SOS)
- Managing authentication (JWT)

- Applying business logic (cooldowns, active SOS checks)
- Running escalation logic asynchronously
- Interacting with database

Key logic implemented:

- Prevent duplicate SOS
- Cooldown between SOS requests
- Role-based access (user, volunteer, admin)
- Guest SOS handling using temporary IDs

3.4. Database (MongoDB)

MongoDB is used to store all persistent data.

Collections include:

- Users
- SOS records
- Volunteers

Key features:

- Stores geolocation using GeoJSON
- Uses $near queries to find nearby volunteers
- Tracks SOS lifecycle:
    - createdAt
    - acceptedAt
    - resolvedAt

3.5. Real-Time Communication (Socket.IO)

Socket.IO enables instant communication between users and volunteers.

How it works:

- When SOS is triggered → backend emits sos-alert
- Volunteers receive alert instantly
- When accepted → sos-accepted event sent
- When resolved → sos-resolved event sent

This ensures:

- No need for page refresh
- Instant updates
- Live synchronization

3.6. Email / Notification Service

Used for escalation and trusted contact alerts.

Functions:

- Sends email to trusted contacts
- Sends alert to admin if SOS remains unresolved

This acts as a **fallback mechanism** when real-time volunteer response fails.

3.7. Asynchronous Processing Layer

One of the most important parts of your system.

How it works:

- After SOS is triggered, backend immediately responds to frontend
- Meanwhile, escalation logic continues in background using setTimeout

Flow:

1. Send SOS to nearby volunteers
2. Wait for response
3. If no response:
    - Increase radius
    - Retry
4. After multiple attempts:
    - Send email to admin

This ensures:

- Non-blocking API
- Better performance
- Reliable handling of emergencies

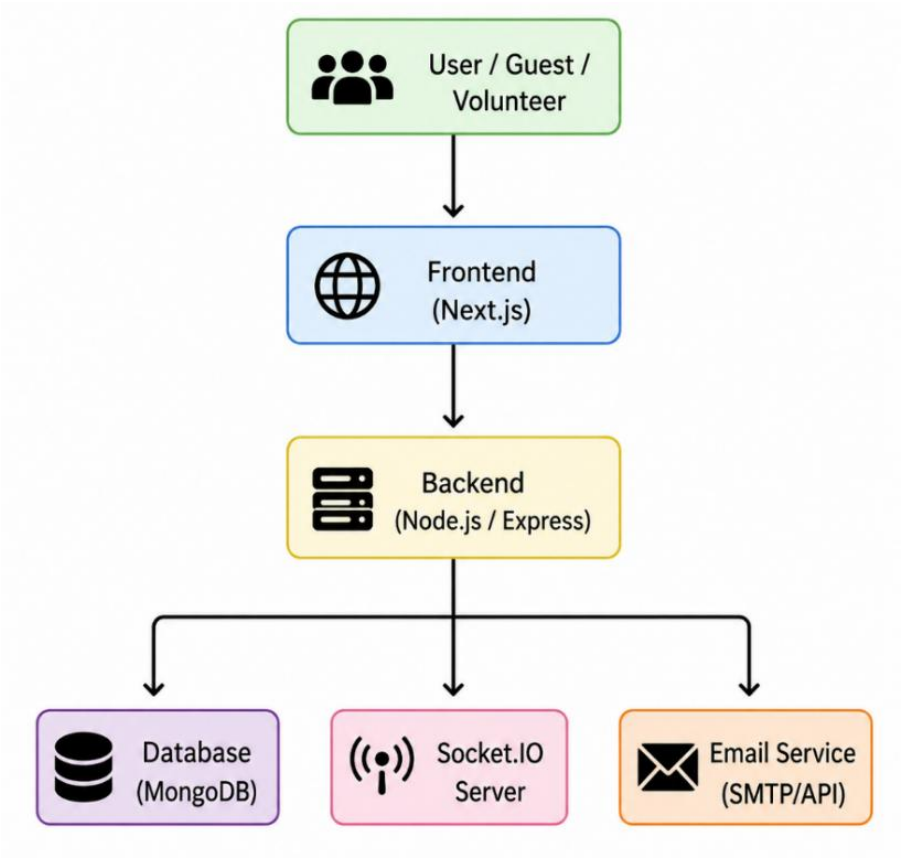


Figure 3.1: System Architecture of SheShield

4. KEY FEATURES

The SheShield system is designed with a focus on real-time responsiveness, reliability, and accessibility during emergency situations. The following features highlight the core functionality and technical strengths of the application.

4.1 Real-Time SOS System

The system enables users and guest users to trigger emergency SOS alerts instantly. Once triggered, the request is processed by the backend and broadcast to nearby volunteers using WebSocket communication.

- SOS requests are created and stored in the database with location and timestamp
- Real-time alerts are emitted using Socket.IO without page refresh
- Volunteers receive instant notifications with options to accept or decline
- The system ensures minimal delay between SOS trigger and alert delivery

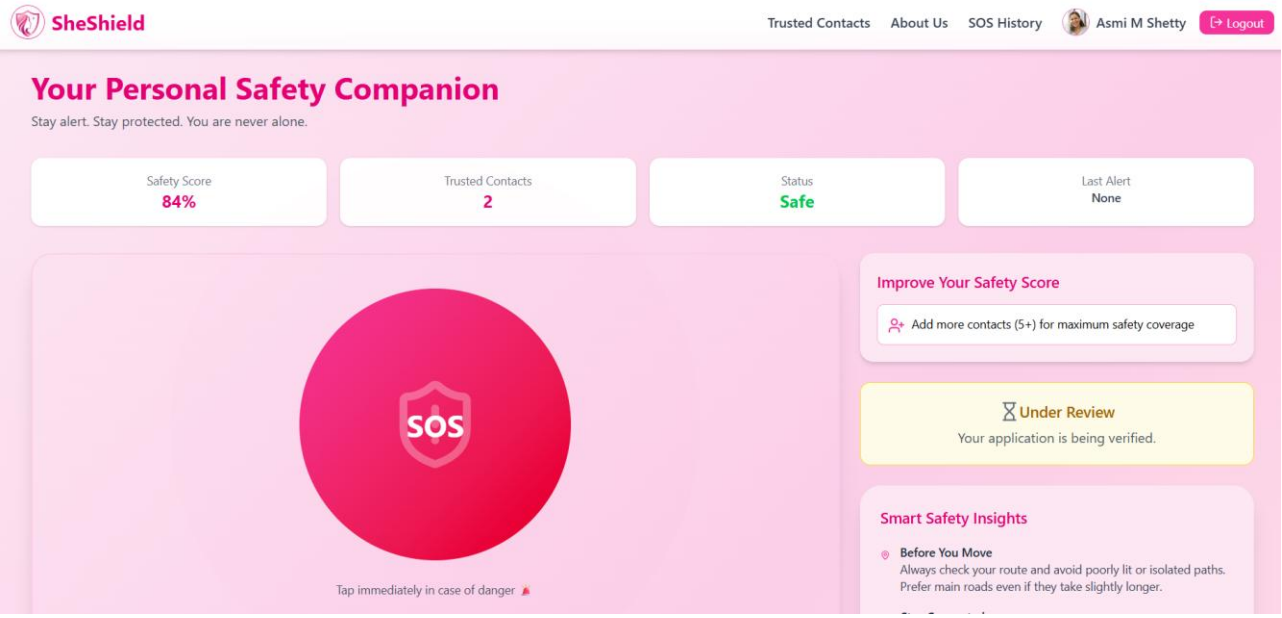


Figure 4.1: SOS button interface (user side)

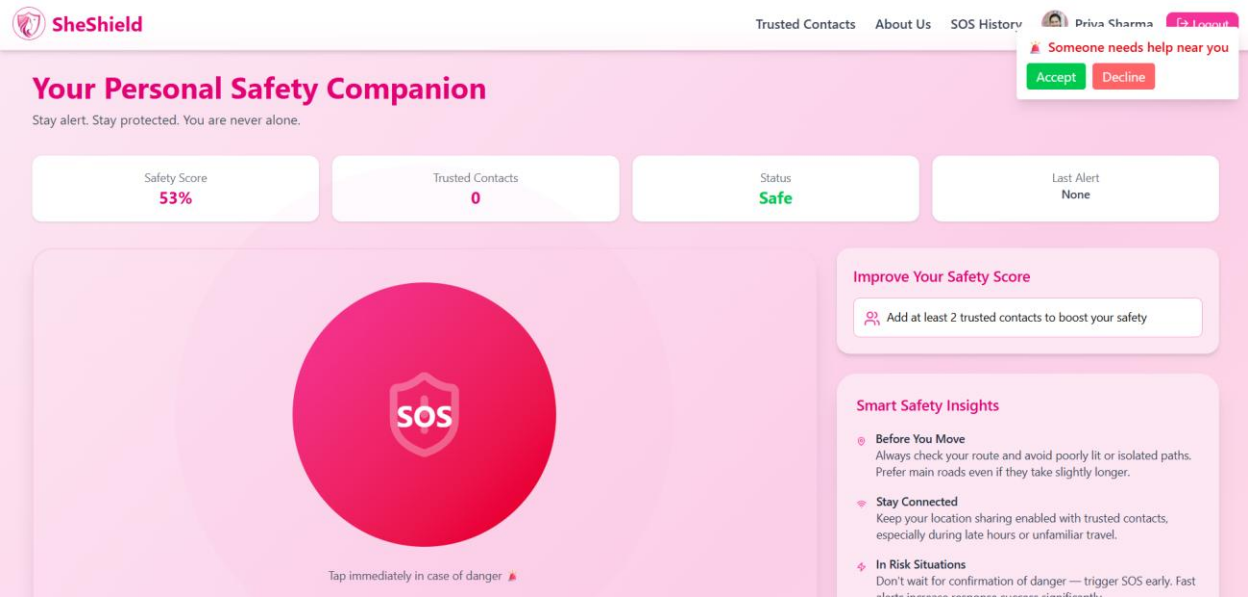


Figure 4.2: Toast/notification received by volunteer

4.2 Guest SOS Support

To ensure accessibility during critical situations, the system allows non-registered users to trigger SOS alerts without authentication.

- A temporary guest ID is generated and stored on the client side
- This ID is used to track the SOS session
- Guests can view active SOS status and responding volunteers
- Eliminates the need for login during emergencies, reducing friction

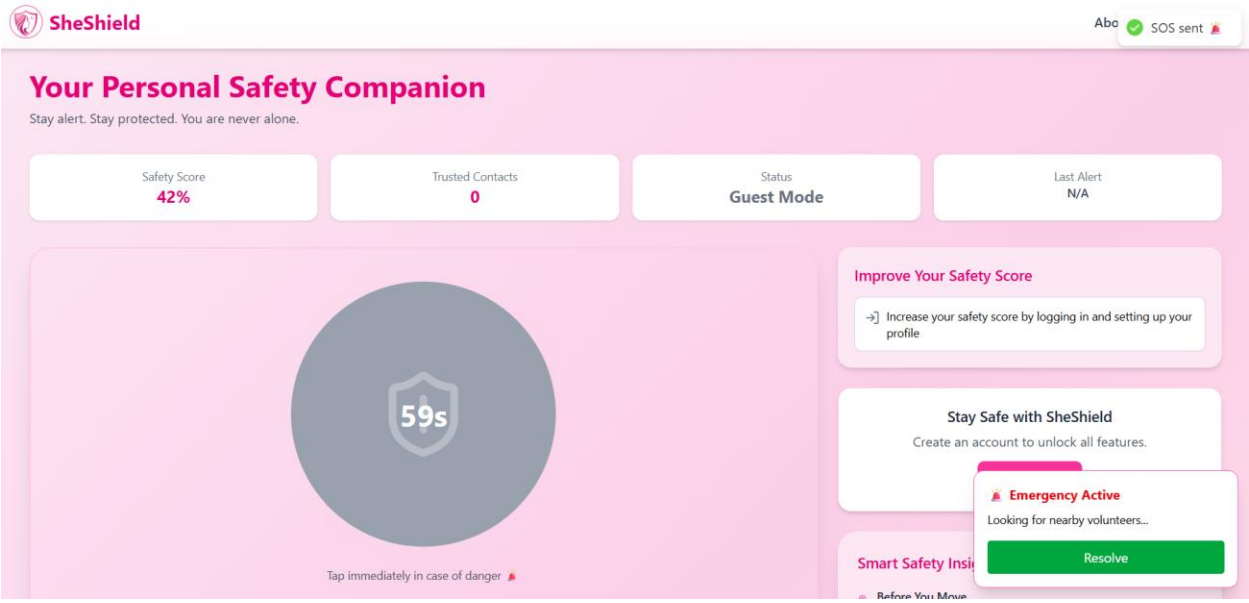


Figure 4.3: Guest triggering SOS (without login)

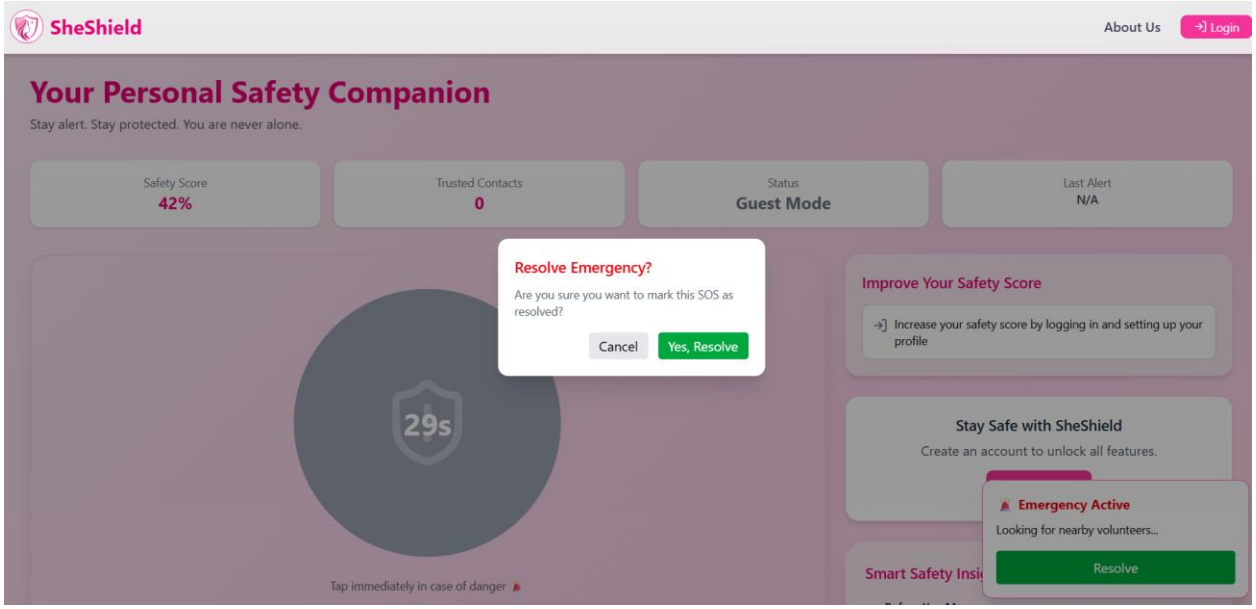


Figure 4.4: Guest SOS panel resolving

4.3 Geolocation-Based Matching

The system uses geospatial queries to identify the nearest available volunteers.

- MongoDB $near query is used for distance-based filtering
- Coordinates are stored using GeoJSON format
- Volunteers within a defined radius (e.g., 5km) are selected
- Ensures faster response by prioritizing proximity

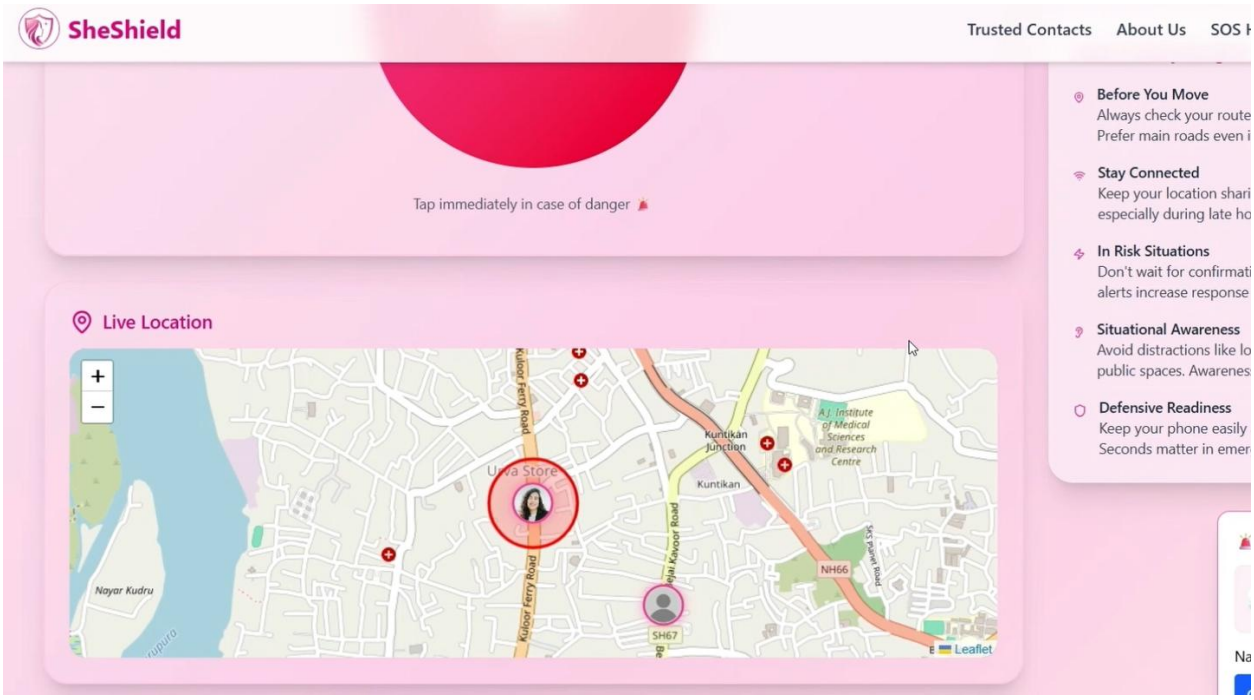


Figure 4.5: Map showing user location and nearby volunteers

4.4 Volunteer System

The platform includes a structured volunteer management system to ensure reliability and trust.

- Users can apply to become volunteers by submitting personal details
- Applications are stored and marked as pending
- Admin verifies and approves or rejects applications
- Approved users are upgraded to volunteer role
- Volunteers can accept SOS requests and assist users

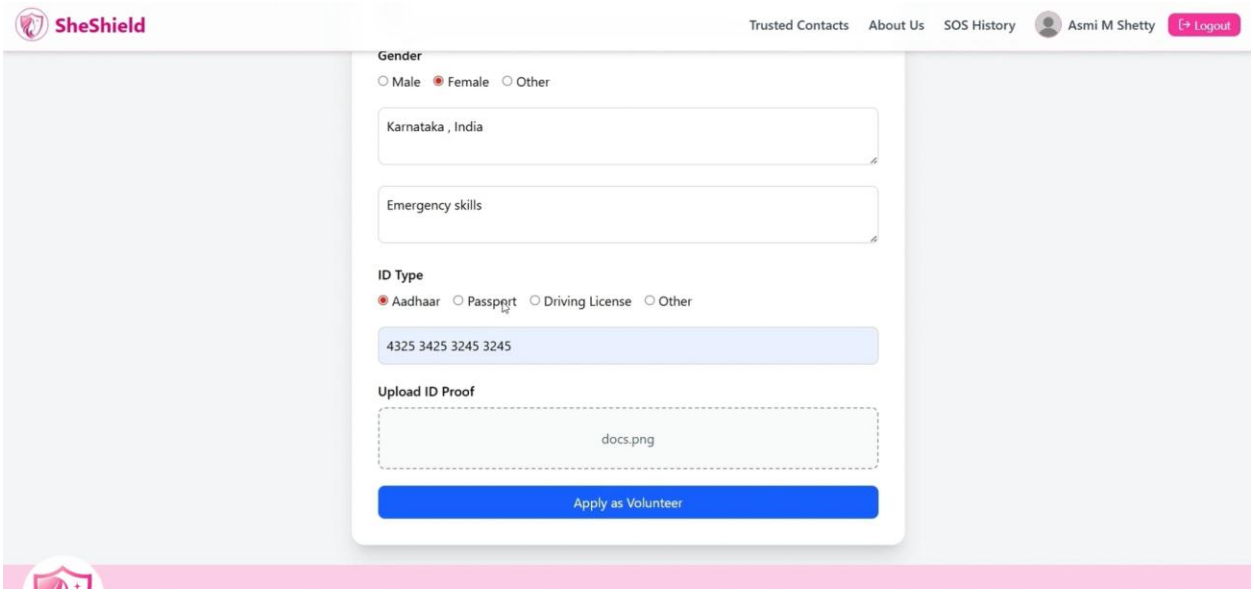


Figure 4.6: Volunteer application form

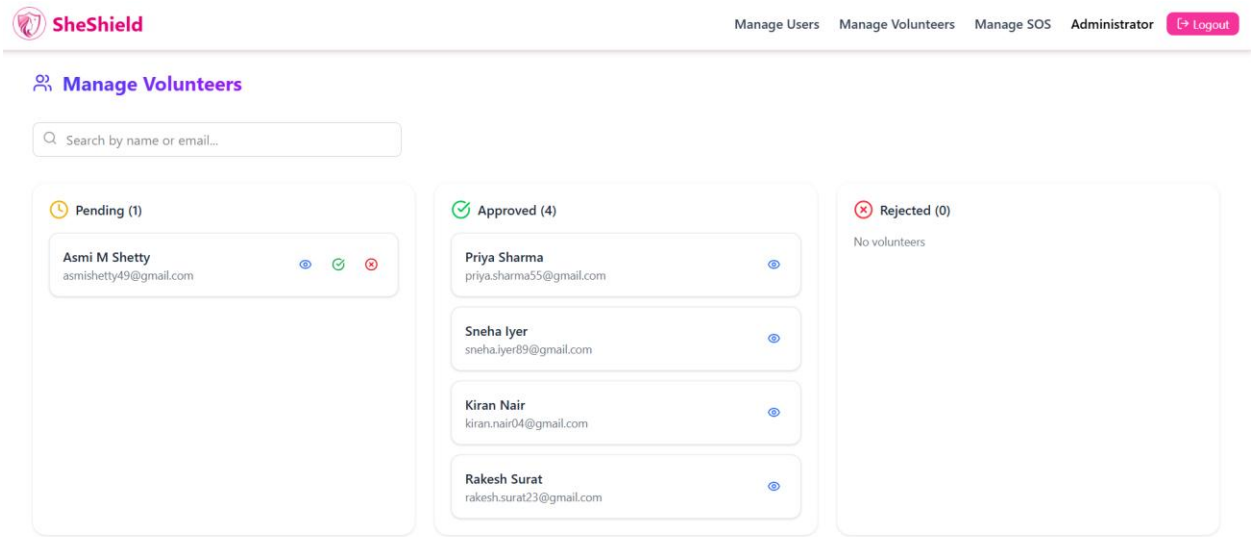


Figure 4.7: Admin approval panel

4.5 Escalation Mechanism

The system implements an intelligent escalation strategy to handle unresponsive situations.

- Initial SOS is sent within a 5 km radius
- If no volunteer responds within a fixed time:
  - The system retries up to 3 times
  - Each retry increases the search radius (e.g., 10 km, 20 km)
- If still unresolved:
  - The system automatically sends an alert to the administrator via email

This process runs asynchronously in the backend, ensuring the frontend remains responsive.

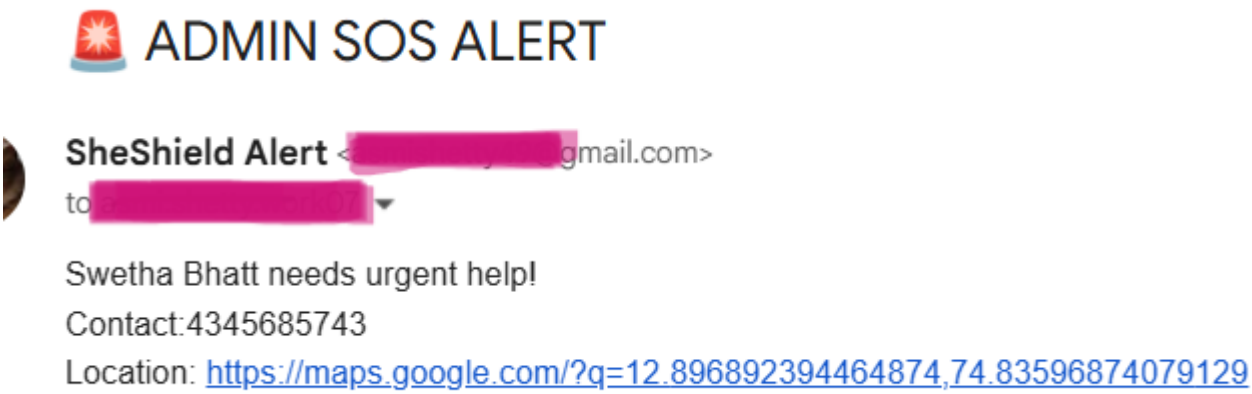


Figure 4.8: Admin email alert (last stage of escalation)

4.6 Trusted Contacts Alert

To increase reliability, the system notifies pre-added trusted contacts during emergencies.

- Users can add multiple trusted contacts (email/phone)
- When SOS is triggered:
    - Email alerts are sent with location details
- Provides an additional safety layer beyond volunteers

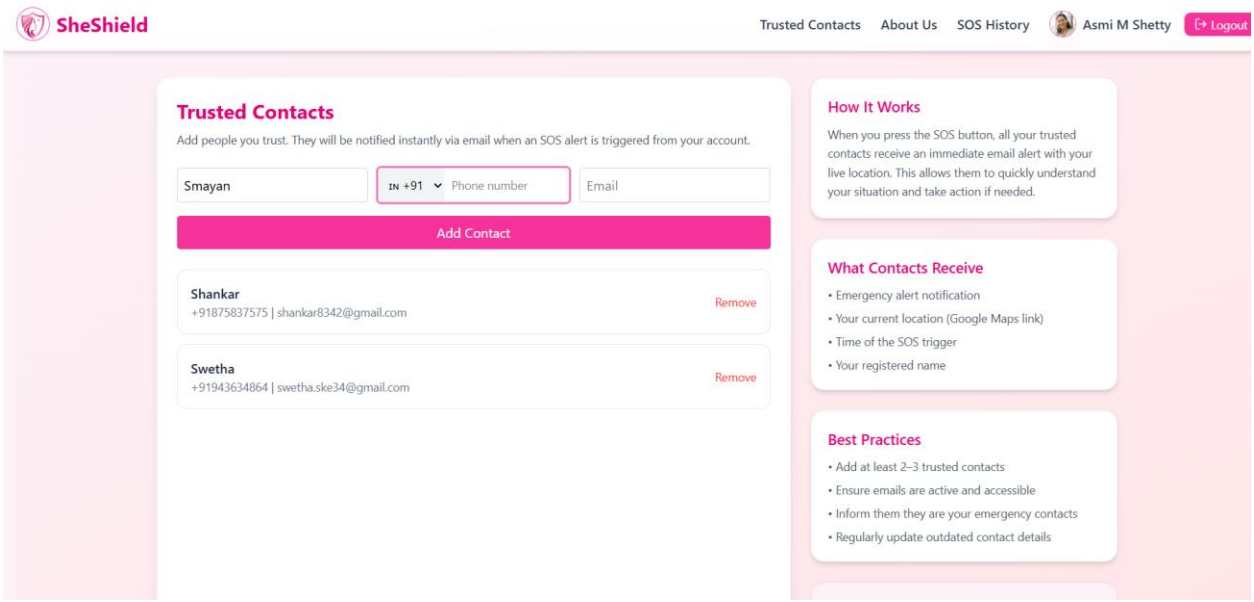


Figure 4.9: Screenshot of adding trusted contacts

Figure 4.10: Example of email alert

4.7 Real-Time Status Tracking

The system provides live updates to both the sender and volunteers.

- Sender can see:
  - List of volunteers who accepted the SOS
  - Their names and contact options
- Volunteers can see:
  - Victim details and location
  - Navigation option via map
- Updates such as acceptance and resolution are pushed instantly using socket events

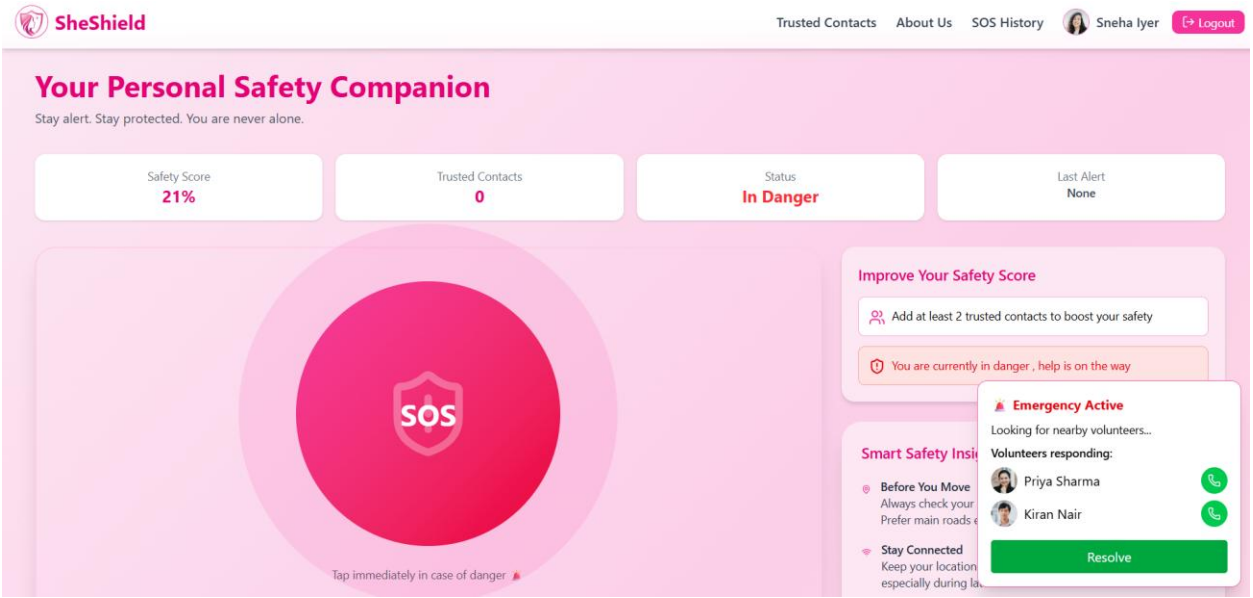


Figure 4.11: Sender dashboard showing responding volunteers

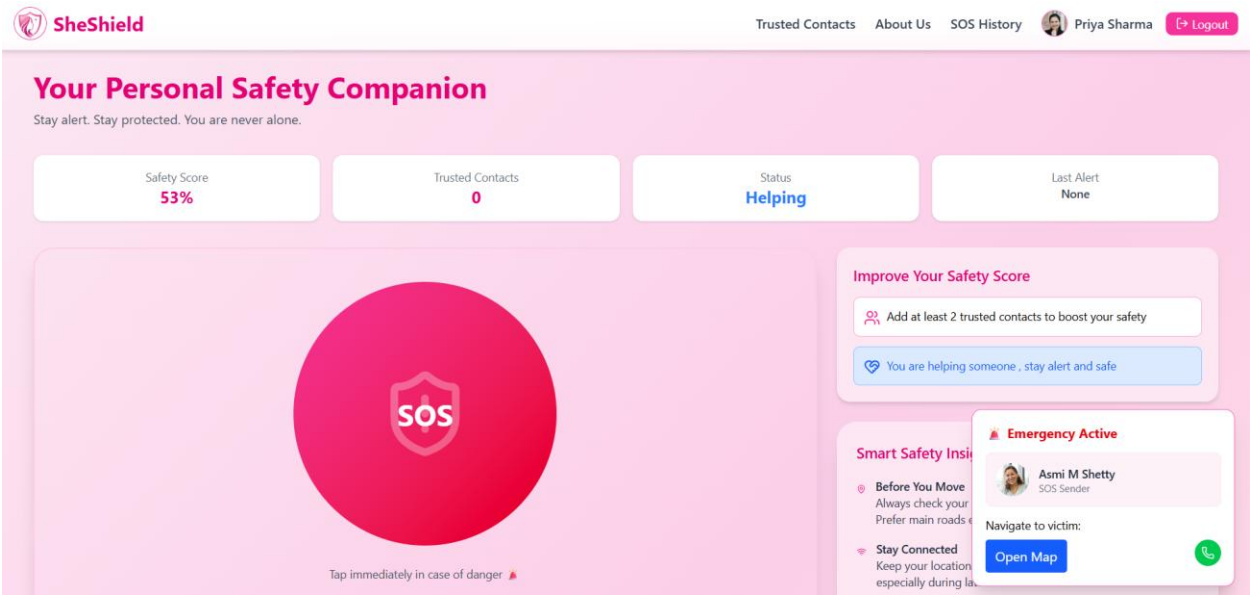


Figure 4.12: Volunteer view showing victim details

4.8 Admin Panel

An administrative dashboard is implemented for system monitoring and control.

- View all registered users and volunteers
- Approve or reject volunteer applications
- Suspend or delete users
- Monitor SOS activity and system statistics
- Access data such as:
    - Total users
    - Active SOS requests
    - Volunteer participation

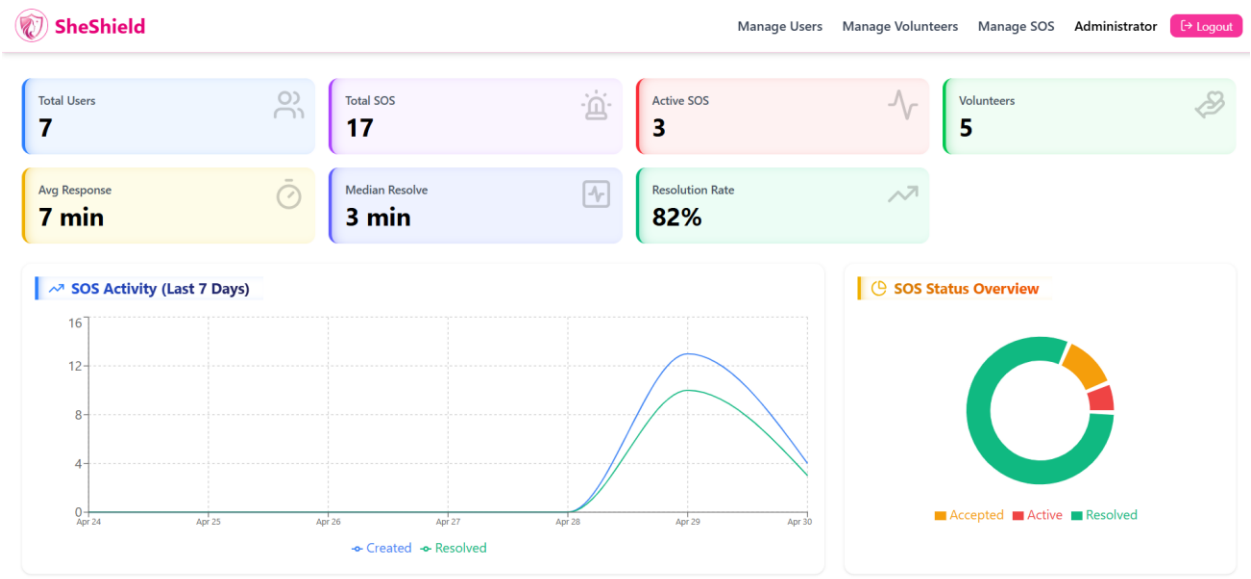


Figure 4.13.1: Admin dashboard screenshot/ Statistics/analytics panel

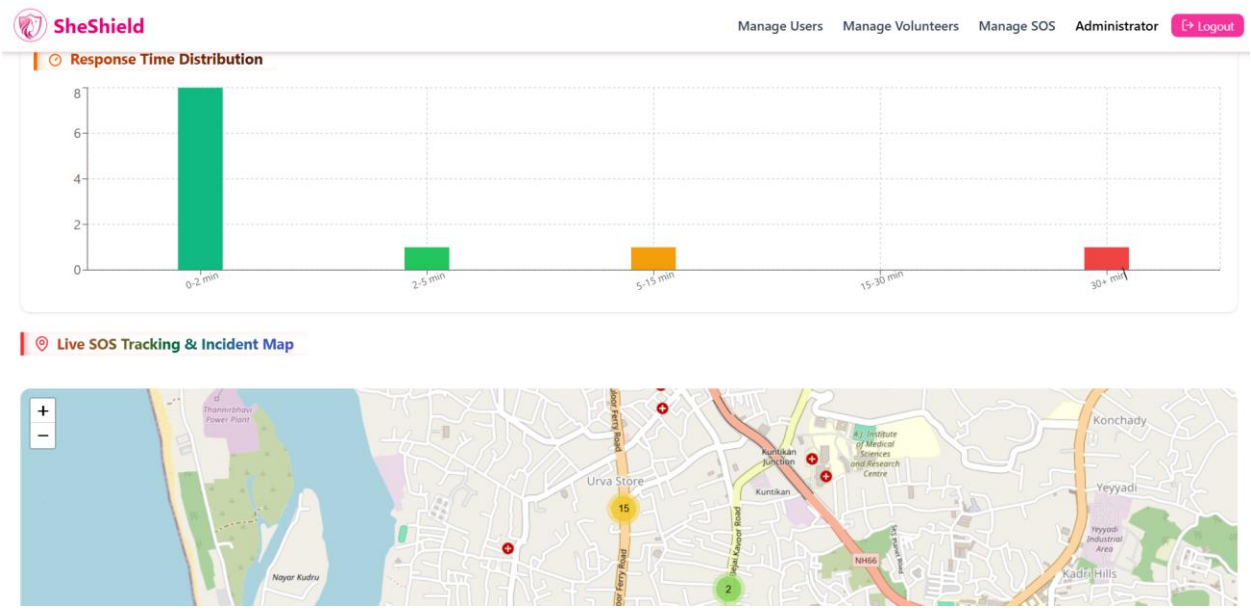


Figure 4.13.2: Admin dashboard screenshot/ Statistics/analytics panel

5. WORKFLOW

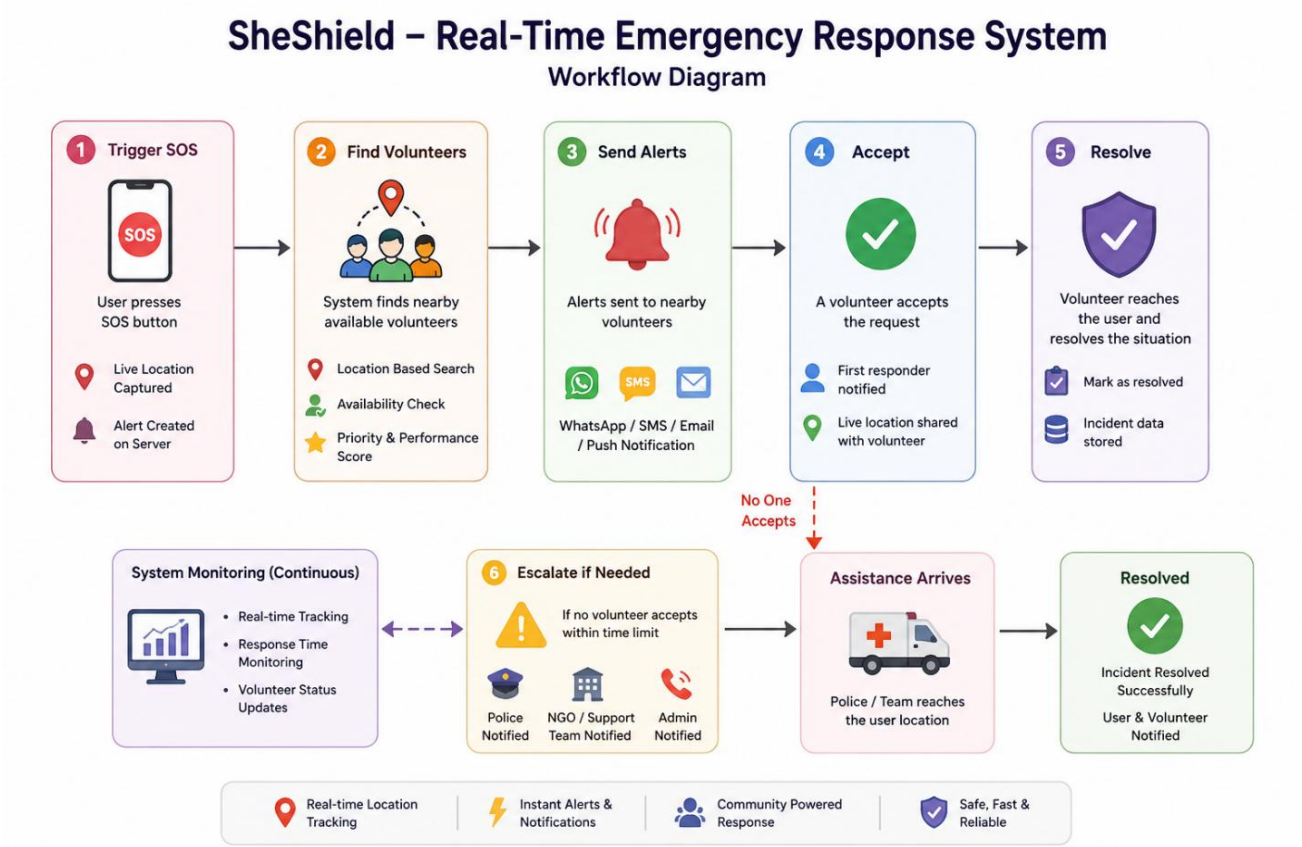


Figure 5.1: SOS Workflow

6. DATABASE DESIGN

The SheShield system uses **MongoDB** as a NoSQL database, with **Mongoose** as the Object Data Modeling (ODM) tool. The database is designed to efficiently handle real-time SOS operations, user management, and volunteer coordination.

The system primarily consists of three collections:

- **User**
- **SOS**
- **Volunteer**

6.1 User Collection

The User collection stores all registered users, including normal users, volunteers, and administrators.

Key Fields:

- name, email, password: Basic user credentials
- userType: Defines role (user, volunteer, admin)
- econtact: Emergency contact number
- isSuspended: Controls account status
- activeSOS: References current SOS
- trusted: Array of emergency contacts
- location: Stores geolocation using GeoJSON

Special Design Features:

- Uses **2dsphere indexing** for fast geolocation queries
- Supports **role-based access control**
- Stores **trusted contacts** for emergency alerts

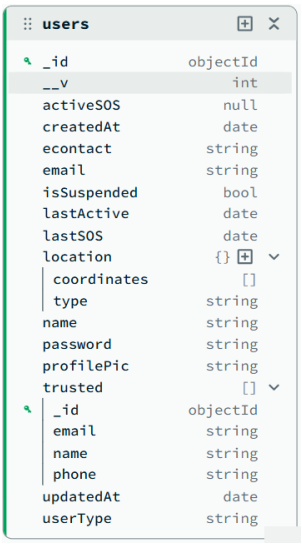


Figure 6.1: User Schema Structure

6.2 SOS Collection

The SOS collection is the core of the system, storing all emergency requests.

Key Fields:

- userId: Reference to the user (null for guests)
- isGuest: Identifies guest SOS
- location: GeoJSON coordinates of emergency
- status: Tracks lifecycle (active, accepted, escalated, resolved)
- acceptedBy: Array of volunteers who accepted
- attemptLevel: Tracks escalation stage
- acceptedAt, resolvedAt: Time tracking fields

Special Design Features:

- Supports **real-time status tracking**
- Tracks **response time and resolution time**
- Enables **multi-volunteer participation**
- Designed for **escalation logic implementation**

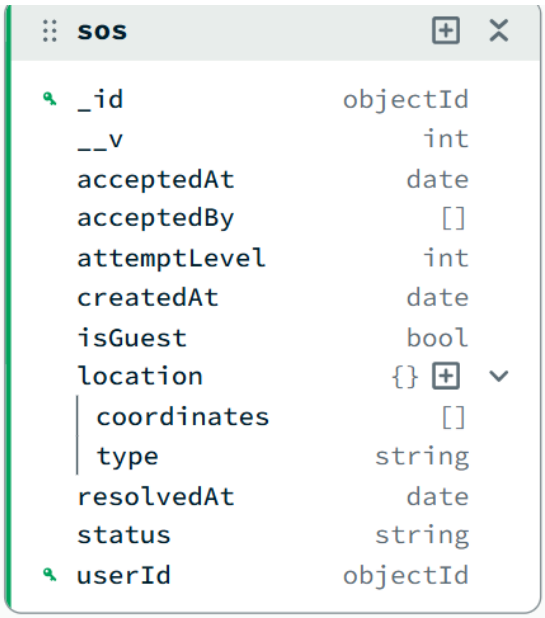


Figure 6.2: SOS Schema Structure

6.3 Volunteer Collection

This collection manages volunteer applications and verification.

Key Fields:

- userId: Links volunteer to user account
- name, phone, age, gender: Personal details
- skills, address: Additional information
- idType, idNumber, document: Verification details
- status: Approval state (pending, approved, rejected)
- isVerifiedVolunteer: Boolean verification flag

Special Design Features:

- Separate collection ensures **clean separation of roles**
- Enables **admin verification workflow**
- Stores **identity proof for security**

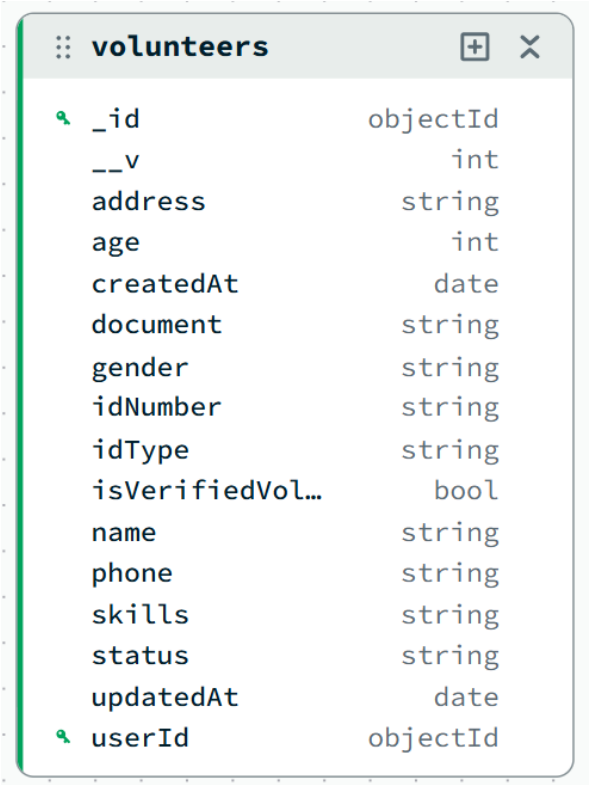


Figure 6.3: Volunteer Schema Structure

6.4 Relationships Between Collections

The system maintains relationships between collections using references:

- A **User** can:
  - Trigger multiple SOS requests
  - Have one active SOS at a time
  - Apply to become a volunteer
- An **SOS**:
  - Belongs to one user (or guest)
  - Can be accepted by multiple volunteers
- A **Volunteer**:
  - Is linked to a User
  - Must be approved before becoming active

Relationship Summary:

- User → SOS (One-to-Many)
- SOS → Volunteers (Many-to-Many)
- User → Volunteer (One-to-One)

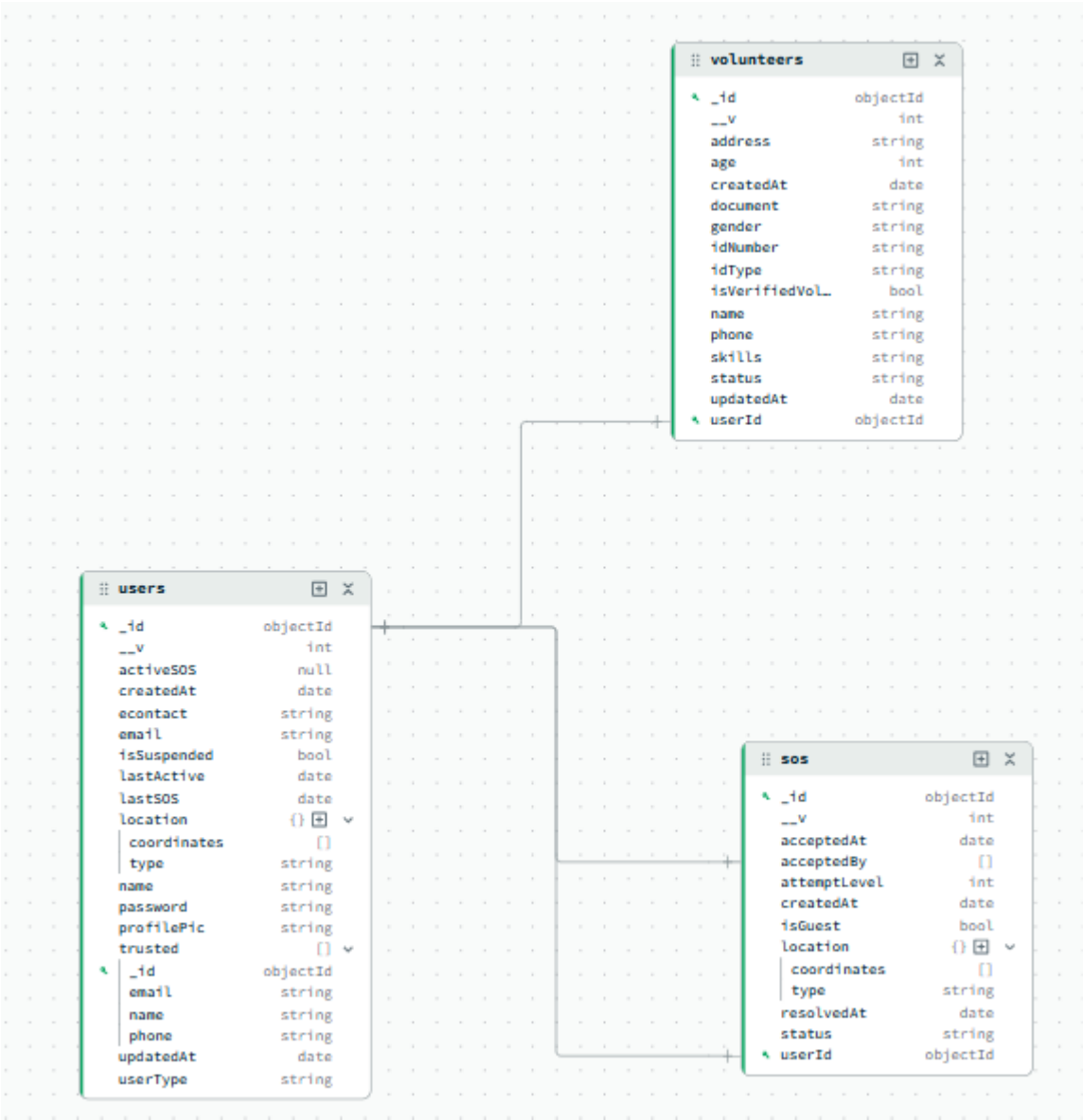


Figure 6.4: Data Model Diagram

6.5 Database Design Advantages

- Efficient geolocation queries using **2dsphere index**
- Flexible schema design using MongoDB
- Optimized for **real-time operations**
- Supports **scalable SOS handling**
- Enables **detailed analytics (response time, resolution time)**

7. IMPLEMENTATION DETAILS

The SheShield system is implemented using a modular and event-driven architecture. The workflow is divided into multiple stages, from SOS triggering to resolution, ensuring real-time responsiveness and reliability.

7.1 User Registration and Authentication Flow

1. The user registers using name, email, password, and emergency contact.
2. Input validation is performed (email format, password strength).
3. Password is hashed using Bcrypt before storing in the database.
4. A JWT token is generated upon successful registration/login.
5. The token is used for authentication in protected API routes.

Key Outcome:
Secure user authentication and role-based access control.

7.2 SOS Trigger Flow (User)

1. Authenticated user clicks the SOS button.
2. Frontend captures current location (latitude, longitude).
3. Request is sent to backend API /triggerSOS.
4. Backend performs:
    - Cooldown check (prevents spam)
    - Active SOS check (only one active SOS allowed)
5. A new SOS entry is created in the database.
6. User's activeSOS field is updated.
7. Trusted contacts are notified via email (non-blocking).
8. SOS alert is emitted to nearby volunteers via Socket.IO.

Key Outcome:
SOS is instantly broadcast while backend continues processing.

7.3 SOS Trigger Flow (Guest User)

1. Guest user accesses the system without login.
2. A temporary guestId is stored in local storage.
3. Guest triggers SOS with location data.
4. Backend validates:
    - Presence of guestId
    - Cooldown restriction
5. SOS is created with isGuest = true.
6. SOS is broadcast to nearby volunteers.

Key Outcome:
Emergency access without authentication.

7.4 Volunteer Matching and Notification

1. Backend queries database using $near geolocation filter.
2. Volunteers within radius (e.g., 5 km) are selected.
3. Online volunteers are filtered using socket connections.
4. Each volunteer receives a real-time SOS alert.
5. Alert includes:
    - Location
    - Sender info
    - Emergency contact

Key Outcome:
Only relevant and available volunteers are notified.

7.5 SOS Acceptance Flow

1. Volunteer clicks “Accept” on the SOS alert.
2. Request is sent to /acceptSOS/:id.
3. Backend:
    - Adds volunteer to acceptedBy array
    - Updates status to “accepted”
    - Sets acceptedAt timestamp
4. Volunteer’s activeSOS is updated.
5. Sender is notified in real-time via socket event.

Key Outcome:
Real-time connection established between sender and volunteer.

7.6 Real-Time Tracking Flow

1. Sender dashboard updates automatically when volunteers accept.
2. Volunteer sees:
    - Sender details
    - Navigation option
3. All updates (accept, resolve) are pushed using Socket.IO.
4. No manual refresh is required.

Key Outcome:
Live synchronization between all participants.

7.7 Escalation Mechanism Flow

1. After SOS is triggered, backend starts an asynchronous process.

2. System waits for a fixed response window.
3. If no response:
    - Retry 1: Increase radius (e.g., 10 km)
    - Retry 2: Increase radius further (e.g., 20 km)
4. If still no response:
    - SOS status set to “escalated”
    - Email sent to administrator
5. Process stops if:
    - SOS is accepted
    - SOS is resolved

Key Outcome:
Ensures no SOS is ignored.

7.8 SOS Resolution Flow

1. Sender clicks “Resolve” button.
2. Request sent to /resolveSOS.
3. Backend:
    - Updates status to “resolved”
    - Sets resolvedAt timestamp
4. Clears activeSOS for:
    - Sender
    - All volunteers
5. Sends real-time “resolved” event to all connected users.

Key Outcome:
Clean termination of SOS lifecycle.

7.9 Admin Workflow

1. Admin logs into dashboard.
2. Admin can:
    - View users and SOS records
    - Approve/reject volunteers
    - Suspend/delete users
3. System displays statistics:
    - Total users
    - Active SOS
    - Volunteer count

Key Outcome:
Centralized system control and monitoring.

The system combines:

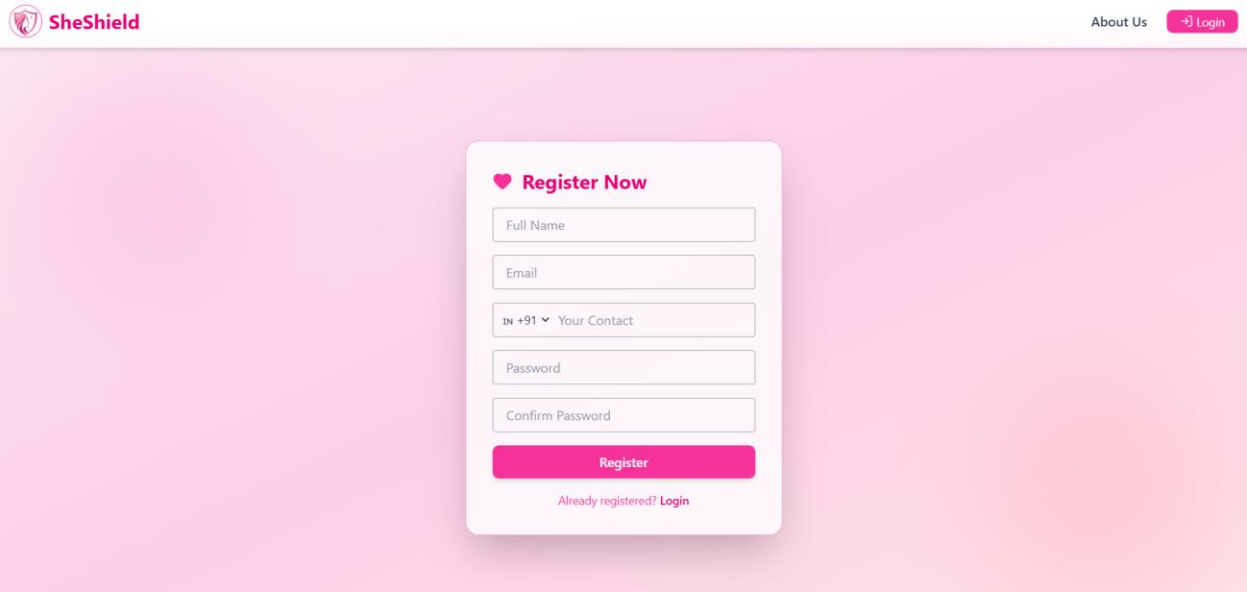
- REST APIs for structured communication
- WebSockets for real-time updates
- Asynchronous backend logic for escalation
- Geolocation queries for intelligent matching

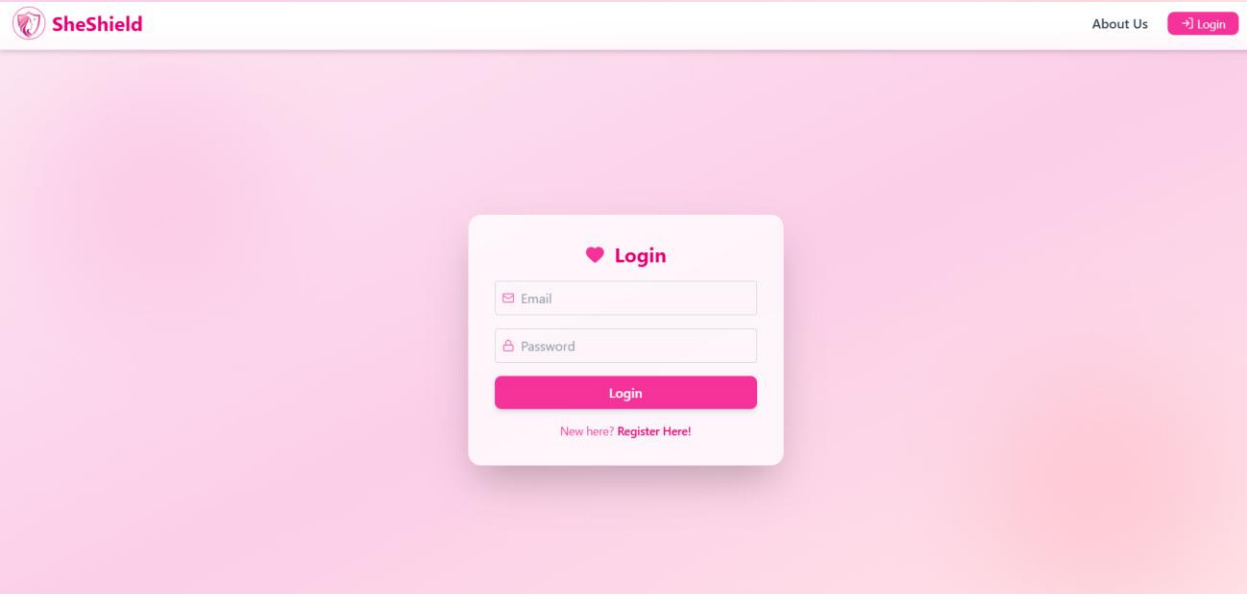
This ensures a **fast, scalable, and reliable emergency response system**.

8. RESULTS / OUTPUT

This section shows the working output of the system with proper screenshots and explanations.

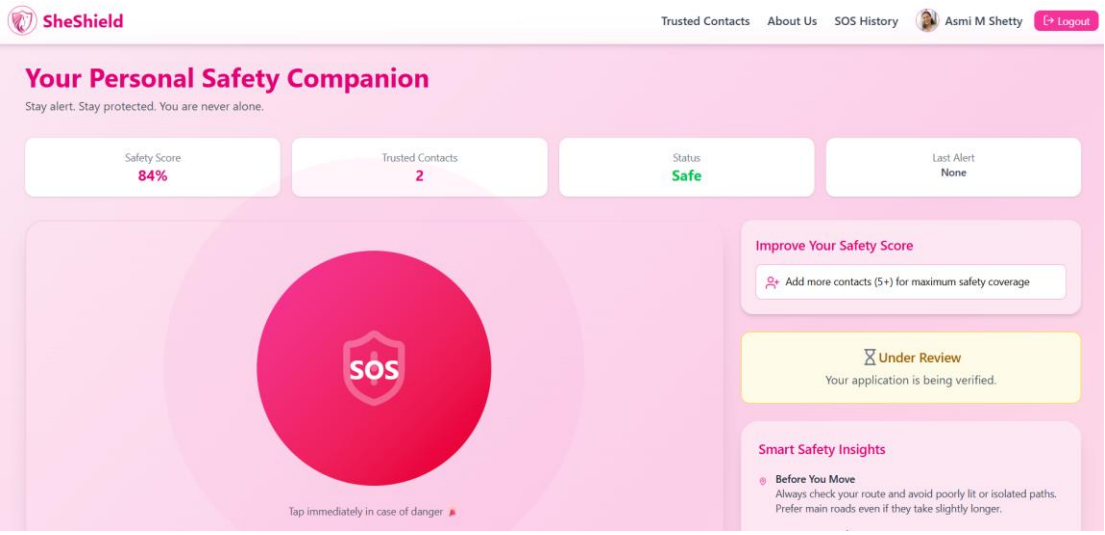
8.1: User Registration / Login Page

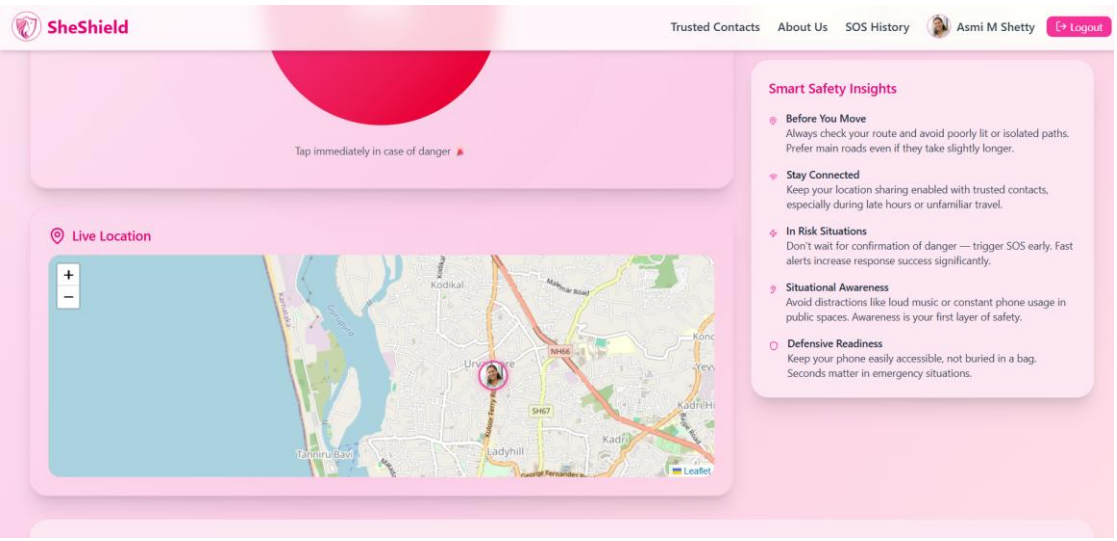


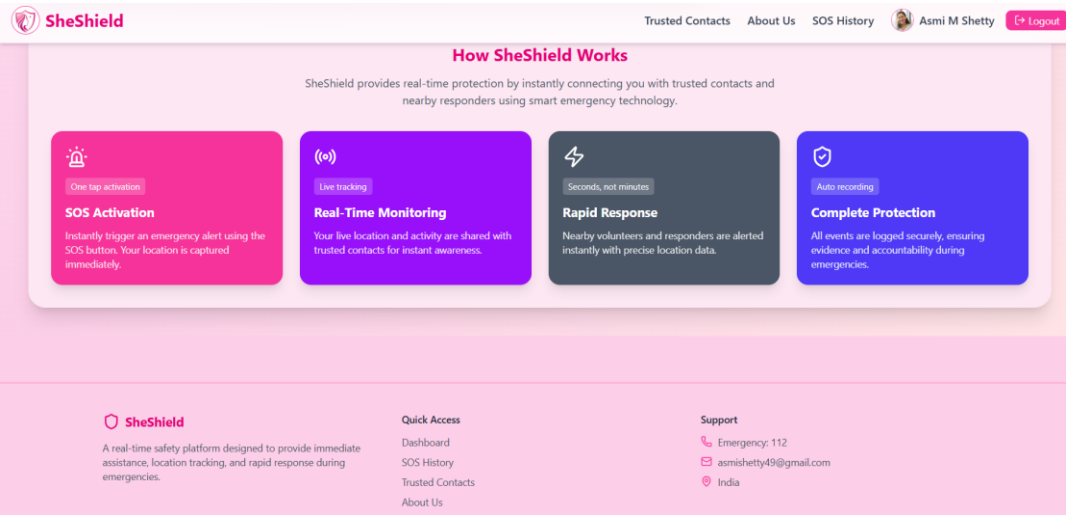


This screen shows the user registration/login page where the user enters credentials to access the system. After successful authentication, the user is redirected to the dashboard.

8.2: User Dashboard

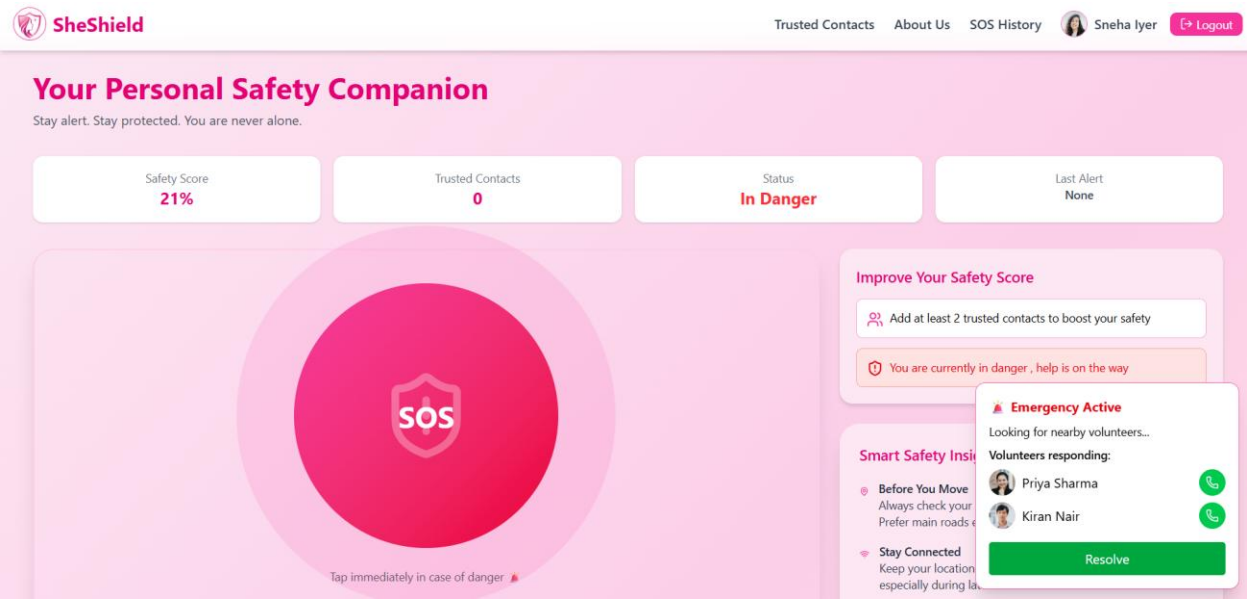

SheShield
Trusted Contacts
About Us
SOS History
Asmi M Shetty
Logout
Your Personal Safety Companion
Stay alert. Stay protected. You are never alone.
Safety Score
84%
Trusted Contacts
2
Status
Safe
Last Alert
None
SOS
Tap immediately in case of danger
Improve Your Safety Score
Add more contacts (5+) for maximum safety coverage
Under Review
Your application is being verified.
Smart Safety Insights
Before You Move
Always check your route and avoid poorly lit or isolated paths. Prefer main roads even if they take slightly longer.


SheShield
Trusted Contacts
About Us
SOS History
Asmi M Shetty
Logout
Tap immediately in case of danger
Live Location
Kodikal
Ladyhill
Leaflet
Smart Safety Insights
Before You Move
Always check your route and avoid poorly lit or isolated paths. Prefer main roads even if they take slightly longer.
Stay Connected
Keep your location sharing enabled with trusted contacts, especially during late hours or unfamiliar travel.
In Risk Situations
Don't wait for confirmation of danger — trigger SOS early. Fast alerts increase response success significantly.
Situational Awareness
Avoid distractions like loud music or constant phone usage in public spaces. Awareness is your first layer of safety.
Defensive Readiness
Keep your phone easily accessible, not buried in a bag. Seconds matter in emergency situations.


SheShield
Trusted Contacts
About Us
SOS History
Asmi M Shetty
Logout
How SheShield Works
SheShield provides real-time protection by instantly connecting you with trusted contacts and nearby responders using smart emergency technology.
One tap activation
SOS Activation
Instantly trigger an emergency alert using the SOS button. Your location is captured immediately.
Live tracking
Real-Time Monitoring
Your live location and activity are shared with trusted contacts for instant awareness.
Seconds, not minutes
Rapid Response
Nearby volunteers and responders are alerted instantly with precise location data.
Auto recording
Complete Protection
All events are logged securely, ensuring evidence and accountability during emergencies.
SheShield
A real-time safety platform designed to provide immediate assistance, location tracking, and rapid response during emergencies.
Quick Access
Dashboard
SOS History
Trusted Contacts
About Us
Support
Emergency: 112
asmishetty49@gmail.com
India

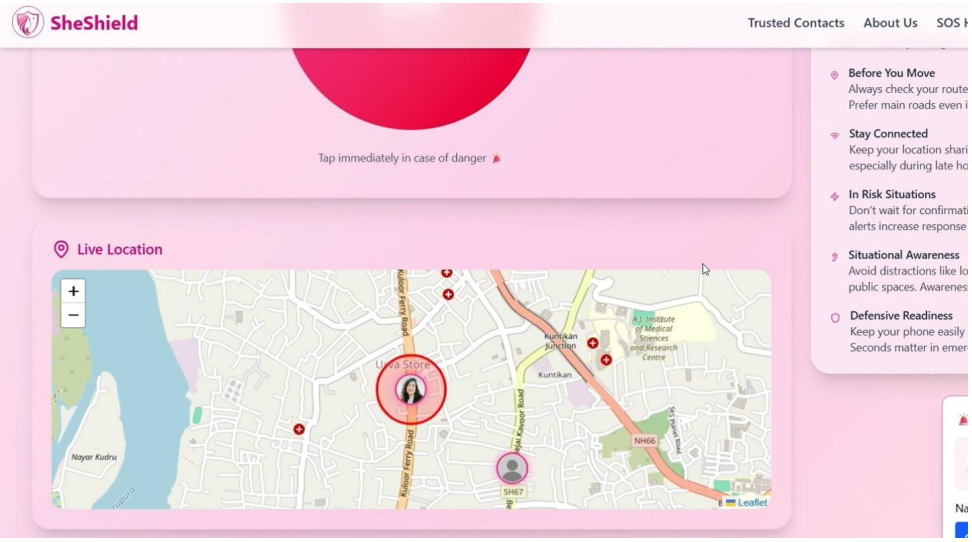
This is the main dashboard of the application where the user can access all features such as SOS trigger, location services, and emergency options.

8.3: SOS Trigger Page / Button



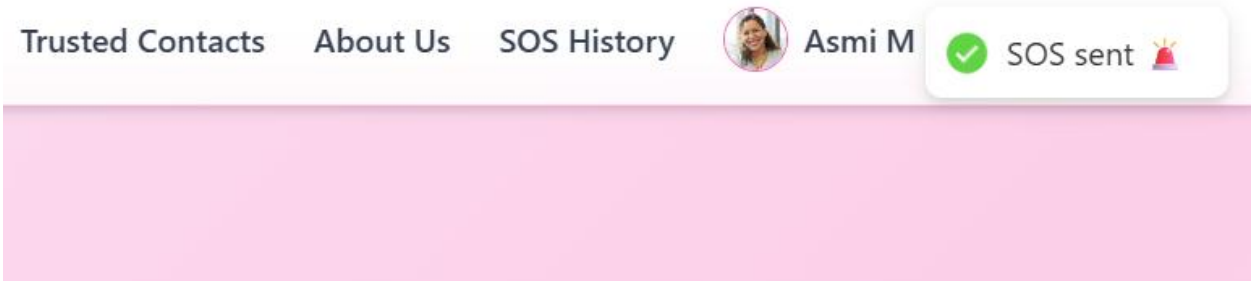
This screen shows the SOS activation feature. When the user presses the SOS button, an emergency alert is triggered and sent to the server/contacts.

8.4: Live Location Sharing (Map View)



This output shows the real-time location tracking of the user. The system captures GPS coordinates and displays them on the map interface.

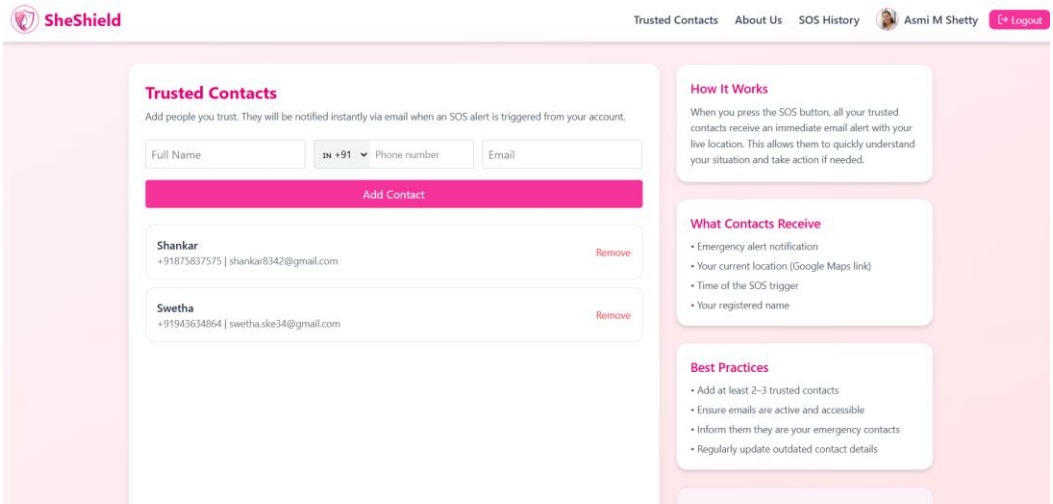
8.5: Emergency Alert Sent Confirmation



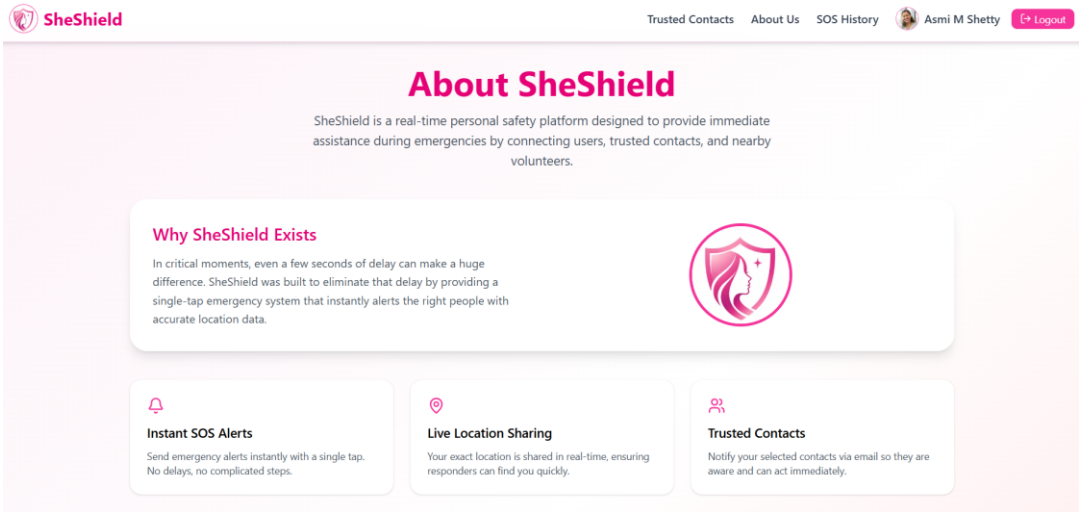
This screen confirms that the SOS alert has been successfully sent. The system notifies emergency contacts/admin with user location details.

8.6: User Pages

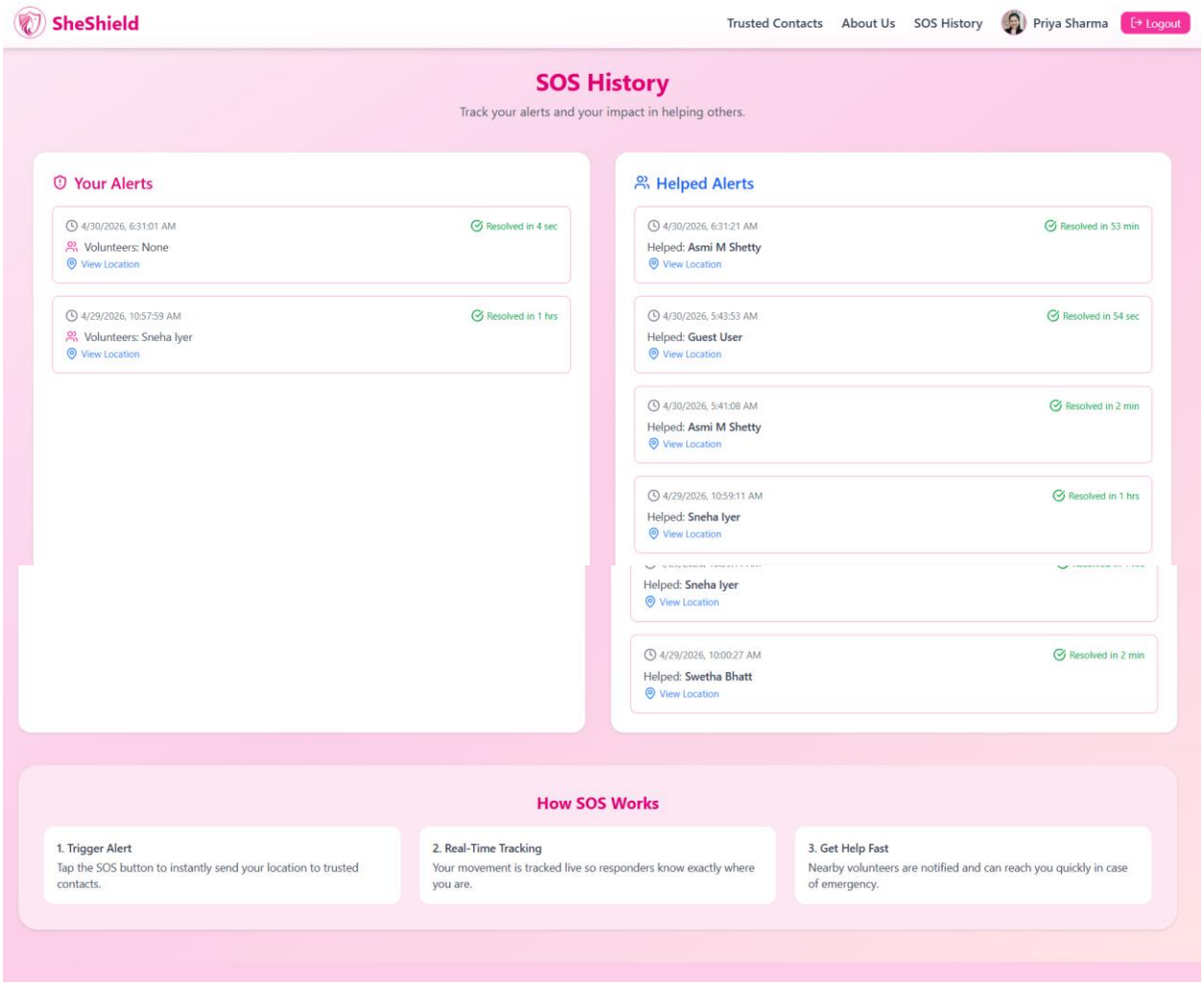
Trusted Contacts page:



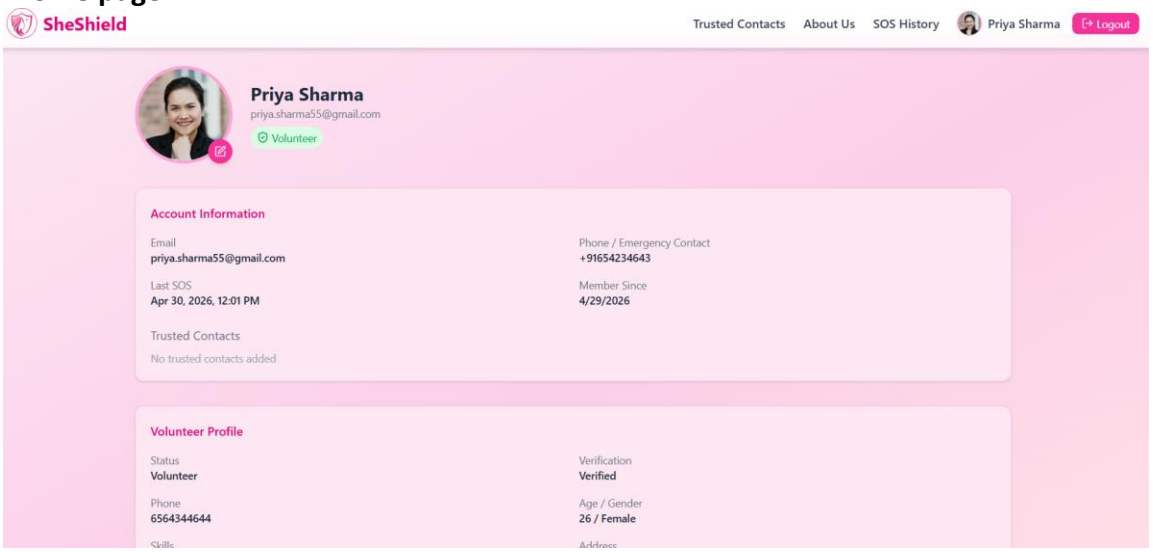
About Us page:

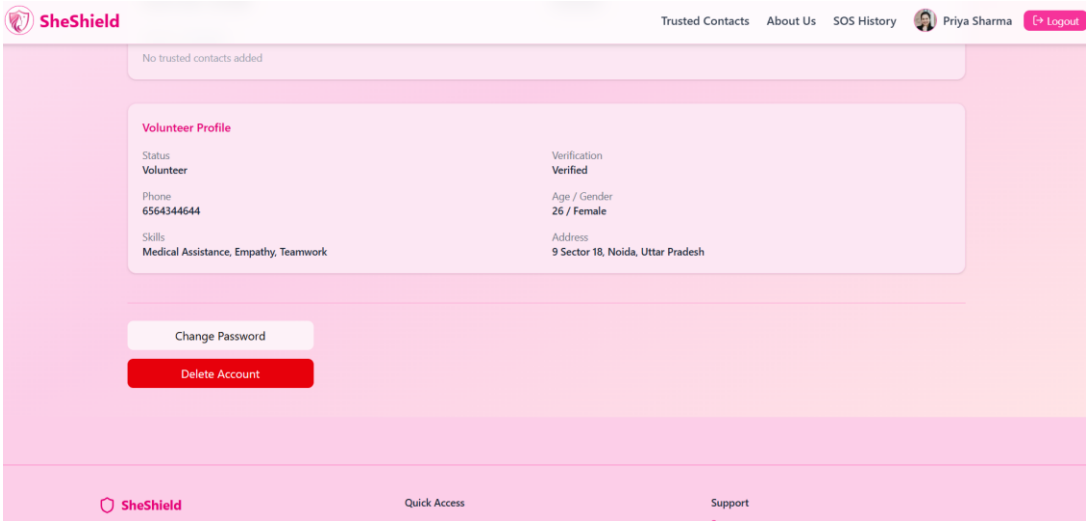


SOS History page:



Profile page:

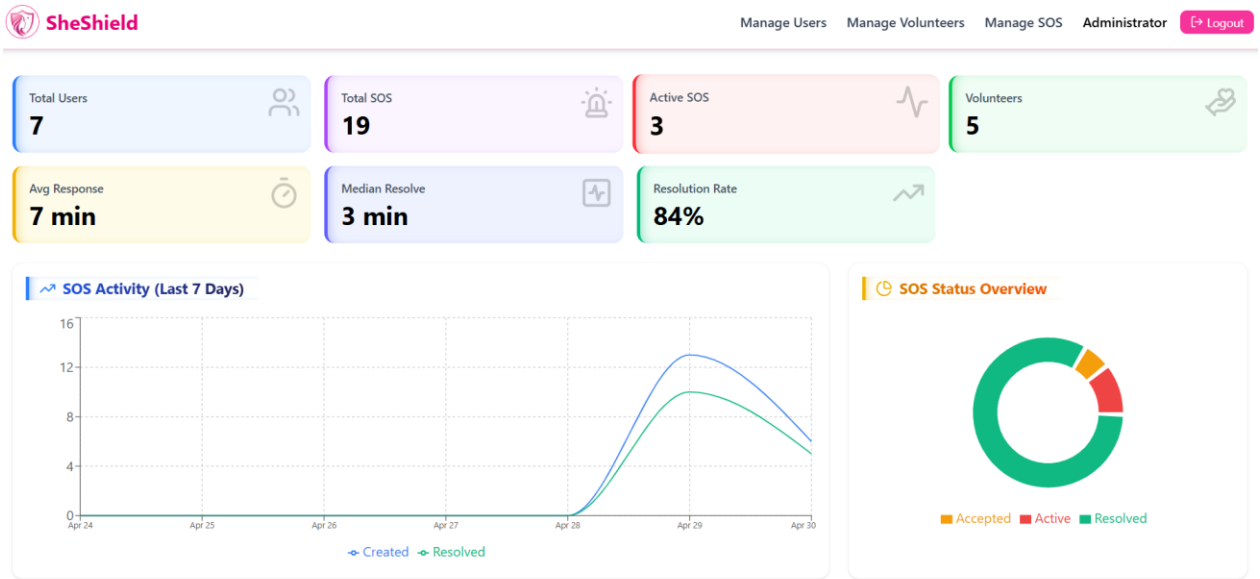


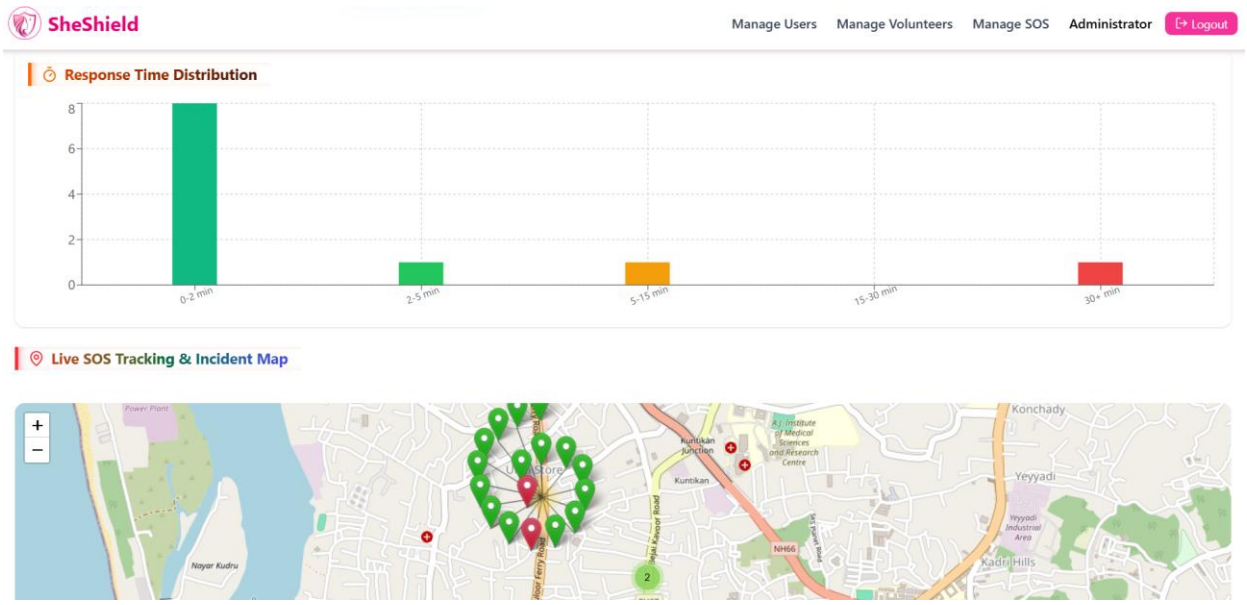


All the above are user pages

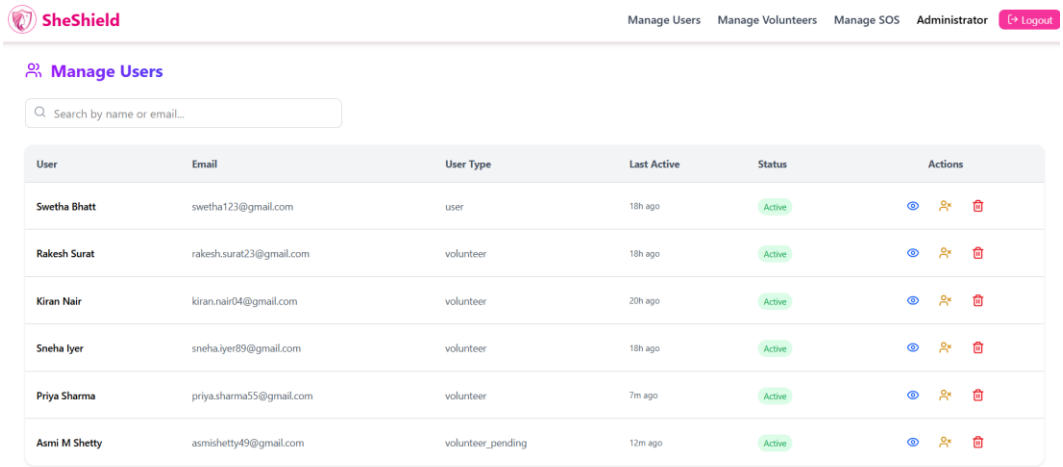
8.7: Admin

Admin Dashboard:

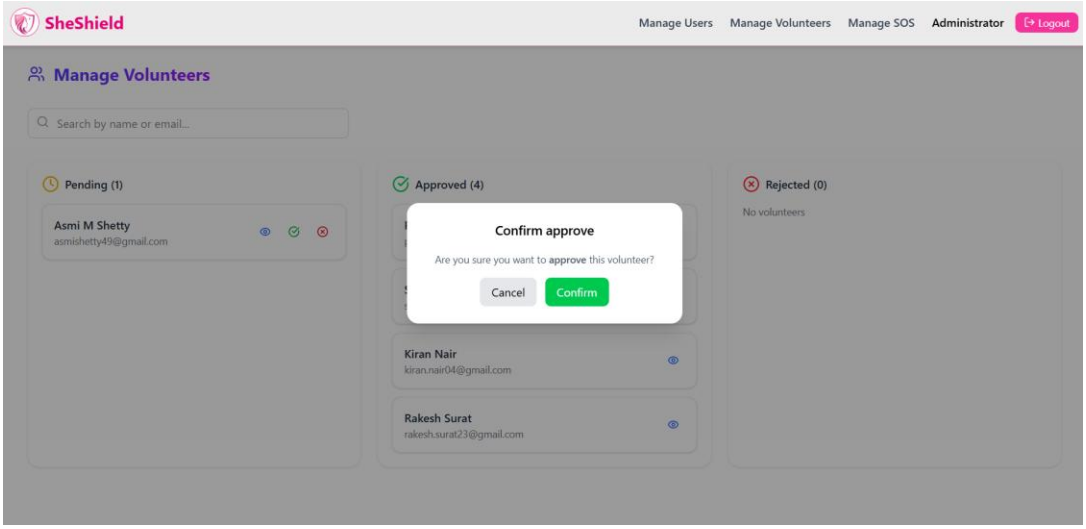




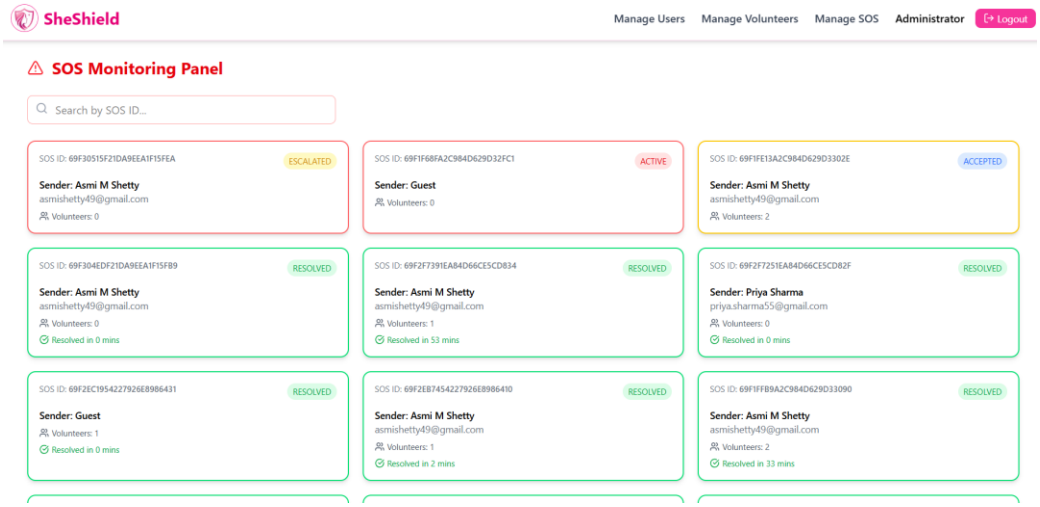
Manage users page:



Manage Volunteers page:



Manage SOS page:



This interface shows the admin pages

9. FUTURE IMPROVEMENTS

The system can be enhanced further with the following improvements:

- **Push Notifications (Firebase Integration):**
  Implement real-time push notifications using Firebase Cloud Messaging to instantly alert users and emergency contacts during SOS activation.
- **Full WhatsApp API Integration:**
  Integrate the WhatsApp Business API to automatically send emergency alerts, location details, and status updates to registered contacts.
- **Live Movement Tracking:**
  Improve location tracking by continuously updating the user's movement in real time instead of static location sharing.
- **Mobile Application Development:**
  Develop a dedicated Android/iOS mobile application for better accessibility, performance, and real-time SOS functionality.
- **AI-Based Priority Detection:**
  Introduce an AI system to analyze SOS requests based on severity, location risk, and user behavior to prioritize emergency responses.

10. CONCLUSION

SheShield successfully demonstrates a scalable, reliable, and real-time emergency response system designed to handle critical situations efficiently. The project emphasizes the importance of rapid communication, where every second matters in emergency scenarios. By integrating real-time technologies such as socket communication, location tracking, and backend event handling, the system ensures that alerts are transmitted instantly without delay.

A key strength of this system is its intelligent escalation mechanism, which ensures that emergency requests are not only generated but also properly routed to the correct responders or contacts. This reduces response time and increases the chances of timely assistance. The system is also designed with reliability in mind, ensuring that even under high load or network instability, SOS requests are not lost and are properly queued or delivered.

Through this project, strong practical exposure was gained in backend development using modern frameworks, handling real-time data streams, and managing database-driven architectures. It also helped in understanding how to design fault-tolerant systems that can continue functioning even when partial failures occur.

Additionally, the project provided valuable experience in system design thinking—especially in areas such as user safety, emergency workflows, and real-world deployment challenges. Overall, SheShield not only serves as a functional emergency response system but also reflects the practical application of software engineering principles in building impactful, real-world solutions.

11. REFERENCES

This project, *SheShield*, is developed using modern full-stack web technologies along with real-time communication and cloud services. The following tools, frameworks, and libraries were used in the development of the system:

Backend Technologies

- **Node.js** – JavaScript runtime environment used to build the server-side application.
- **Express.js** – Web framework used for creating REST APIs and handling server routes.
- **MongoDB (Mongoose)** – NoSQL database used for storing user data, SOS requests, and system logs.
- **Socket.io** – Enables real-time bidirectional communication for instant SOS alerts and live updates.
- **JWT (jsonwebtoken)** – Used for secure authentication and authorization.
- **bcryptjs** – Used for hashing passwords to ensure security.
- **dotenv** – Used for managing environment variables securely.
- **CORS** – Handles cross-origin resource sharing between frontend and backend.
- **express-rate-limit** – Prevents API abuse by limiting repeated requests.
- **nodemailer** – Used for sending email notifications.
- **Twilio API** – Used for sending SMS/WhatsApp alerts in emergency situations.
- **Multer & Cloudinary** – Used for file uploads and cloud-based image storage.
- **validator** – Used for input validation and sanitization.

Frontend Technologies

- **Next.js (React Framework)** – Used for building the user interface and handling routing.
- **React.js** – Core library for building reusable UI components.
- **Tailwind CSS** – Used for styling and responsive design.
- **Socket.io Client** – Enables real-time communication with backend server.
- **Leaflet.js** – Used for interactive maps and location tracking.
- **React-Leaflet & MarkerCluster** – Used for displaying user location and clustering multiple markers.
- **Recharts** – Used for visualizing data in charts and analytics dashboards.
- **React Hot Toast** – Used for showing real-time notifications.
- **Lucide React** – Icon library used for UI enhancement.
- **js-cookie** – Used for managing cookies and session data.

Development Tools

- **VS Code** – Primary code editor.
- **Git & GitHub** – Version control and repository management.
- **Postman** – API testing and debugging.
- **Vercel / Render (Deployment)** – Used for hosting frontend and backend applications.